VIT®

BHOPAL

www.vitbhopal.ac.in

Winter Semester: 2025-2026

Course Code & Name: CSE 0310, Computer Vision

Project Report

Student Name- Mohd.Belal

Registration Number - 23BAI10574

The problem you chose

Smart Retail Theft Detection using Deep Learning and Computer Vision

Shoplifting or retail theft is a significant issue for physical retail stores, and most current methods for catching shoplifters are reactionary, especially with cameras being present everywhere in stores now. Videos from cameras only get examined after shoplifting has already occurred, and not all cameras are watched by security guards or store owners in real time.

Problem Statement: The current method for detecting shoplifters is not proactive, as it relies on looking at videos after the theft has occurred. In order to tackle this, our proposed solution presents a way to monitor all connected cameras in real-time.

Why It Matters

- Retail shrinkage costs Indian retailers 40,000 crores yearly (CII, 2023), which translates to more than 100 billion dollars worldwide.
- Only a fraction of (under 30%) shoplifting incidents are caught in real-time using traditional methods.
- No store owner can afford to hire security personnel to monitor all cameras in real time.
- Small/medium retailers are more affected than any other business scale due to the current economic situation.
- The solution that we propose will give all stores (small or large) security intelligence that was hitherto accessible only to large corporations.
- Apart from the economic implications, shoplifting also affects worker safety.

Your Approach to Solving It

The solution that we have built is a modular system for real-time video analysis, designed in four distinct stages:

- **Stage 1 - Person Detection**

For detecting in each video frame we used YOLOv8n (nano edition) and fine-tuned it to only detect the class "person" in order to avoid false positives. If YOLOv8 is not installed, we use HOG+SVM as a fallback.

- **Stage 2 - Multi-Object Tracking**

We use an IOU-based tracker to maintain a consistent, unique ID for each detected person in each frame. This allows us to store their past location, speed, and movement patterns.

- **Stage 3 - Pose Estimation**

In each video frame, MediaPipe Pose identifies 33 body landmarks for each individual. Calculations on the angles between various joints (knee angle, wrist-to-torso angle, lean, etc.) help us to detect actions like hunching or hiding.

- **Stage 4 - Behavior Analysis**

Using rule-based logic, we analyse each person's historical data and posture to classify actions into LOITERING, CROUCHING, CONCEALMENT, or SUDDEN_MOVEMENT. The system uses a varying severity level for alert messages and a cooldown of 90 frames to avoid repetitive alerts. A CSV file stores the logs. We expose the system via a CLI for server/headless operation and a Streamlit interface for live demonstrations.

Key Decisions You Made

- **Why YOLOv8 was selected over other detectors**

We selected YOLOv8n for our detector due to its COCO mAP of 64.9 and detection time under 5ms, providing us with better accuracy and speed over previous YOLO versions. It was crucial to get a better FPS as we need the system to run at 20FPS on a CPU.

- **Why was rule-based behavior analysis used instead of training a video classifier model?**

Training a video classification model relies on a large amount of video scene data labeled with events of retail theft. This data is generally unavailable. Rule-based systems, on the other hand, are easily interpretable, don't require training data, and can be readily modified.

- **IoU tracking vs. DeepSORT**

As with the choice of a training model for classification, significant quantities of labeled video data of retail theft events are required for DeepSORT's training. Unlike rule-based systems, they require considerable training data and may not be as easy to interpret or modify.

- **Modular pipeline structure**

Each stage of the pipeline is written as a separate class, allowing any component (e.g., detector) to be replaced with another (e.g., a different model or a tracker like DeepSORT) without affecting the rest of the pipeline.

- **Alert cooldown mechanism**

We implemented an alert cooldown period of 90 frames (~3 seconds) for each trackid/alerttype combination. Otherwise, a person loitering for thirty seconds would result in over a hundred similar alerts. This still allows the system to respond to new critical alerts quickly.

Challenges You Faced

Running at real-time FPS on CPU-only hardware

The computational complexity of YOLOv8, combined with pose estimation, is quite high. It was essential to use the smallest possible YOLOv8n and to filter for the "person" class only, and also make pose estimation optional to ensure acceptable FPS. Pose estimation lowers the frame rate from ~25 to ~16.

Reduction of false positives

There were far too many false positives initially. People bending down to pick something up were often categorized as having engaged in concealing and moving at a fast rate. By adjusting thresholds, we ensure that detection of concealment takes 12 consecutive frames rather than just one. A running person was flagged as a sudden movement.

Identity persistence across occlusions

When a person went behind a shelf and came back out, they would be given a new ID by the IOU tracker, and the behaviour timer would be reset. Increasing the g "miss streak" to thirty eliminated these incorrectly identified objects.

Balancing sensitivity and specificity

A lower threshold leads to more false positives, but is also a greater measure of detection. While this frustrated staff, high thresholds would miss more real thefts. Through experiments, we found the right balance for a retail scenario.

What You Learned

Technical learnings:

- The underlying architecture of YOLOv8 and why it achieves real-time detection. How varying confidence thresholds relate to precision and recall.
- How an IoU based tracker works frame to frame, and in what scenarios it would fail (occlusion, rapid movement, and closely related bounding box shapes).
- How the pose skeleton of 33 joints used in MediaPipe can be represented by simply derivable, understandable joint angles using trigonometry.
- How to design an efficient production-ready pipeline, including graceful failure, running headless, and using well-structured logging.

Design learnings:

- Rule-based systems are heavily under-appreciated, are very interpretable, explainable to non-technical users, and easily updatable without the need for data. These factors were all important in this project.
- Modularity should not be viewed as a suggestion, but a requirement that allows systems to be easily iterated and modified through pluggable parts.
- Generating effective alerts that notify the user without annoying them is almost as important as the alerting process itself.

Broader learnings:

- There is a huge ethical consideration in AI surveillance in relation to privacy. The right not to be constantly observed by AIs needs to be balanced with security.
- The real-world data obtained from video recordings is far less precise than the benchmark data normally used; lighting, camera position, and the many different occlusion situations dramatically reduce system accuracy.
- There's a significant difference between a project 'working' for a demo and a project 'working' in store. Robustness has to be equivalent to functioning capability.